Ghid deploy · RAI Companion · producție client

Domeniu: **https://rai-transitix.filumforge.ro**
Branch Git: **TRANSITIX-RAI-PROD** · Environment GitHub: **RAI-PROD** / production
VPS Contabo: **Cloud VPS 4** · 4 vCPU · 8 GB RAM · 100 GB · IP **185.137.122.45**
Document: **12 octombrie 2026** · instalare curată · OCR Mistral cloud

PRODUCȚIE CLIENT | Companion documents | Fără OSRM / GPU | Caddy + systemd + Docker DB

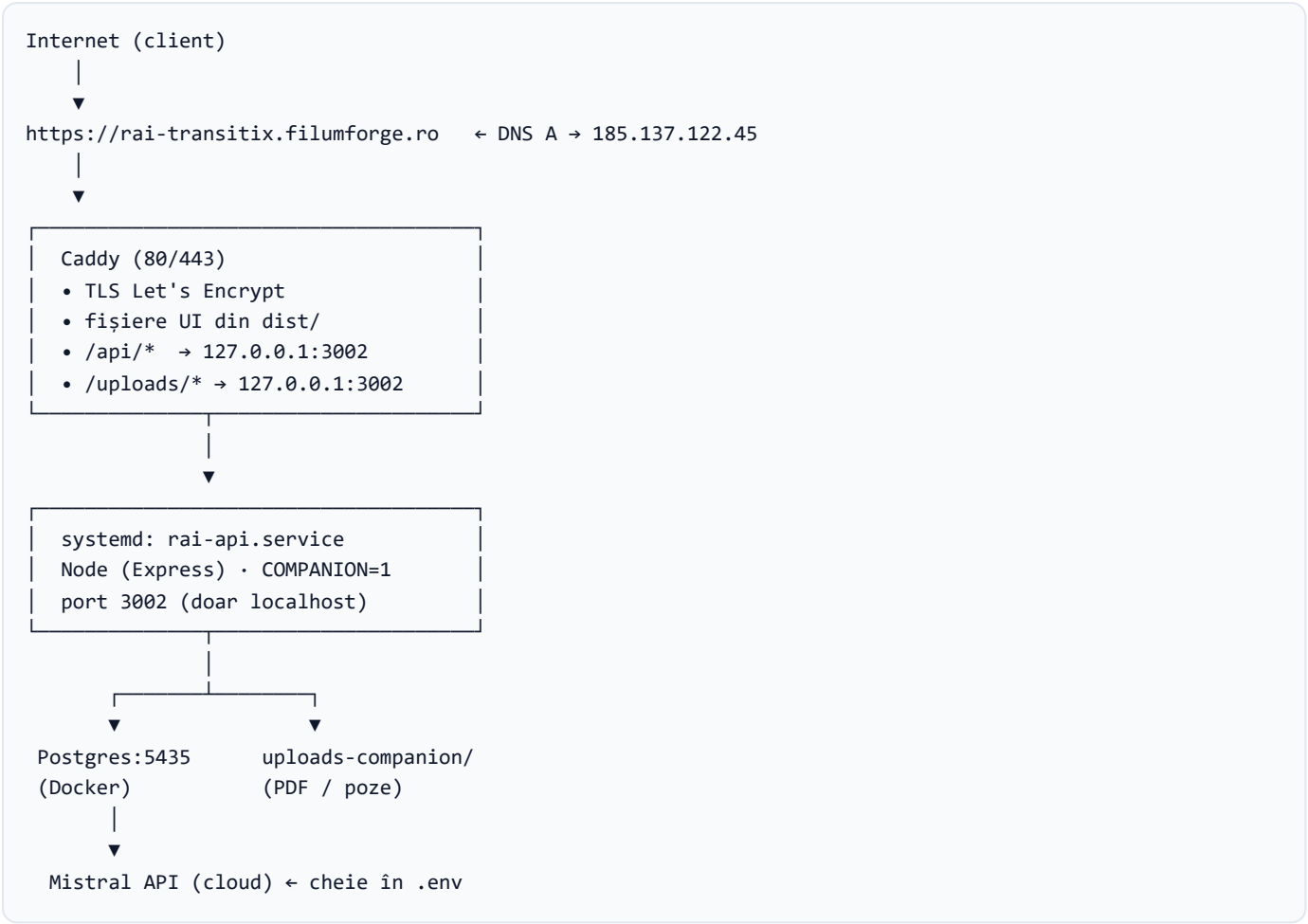
Nu folosi pe VPS `npm run dev:companion` / `dev:companion-api`. Acelea sunt doar pentru PC-ul tău. Pe server: build o dată + servicii care pornesc la boot.

Ce rulează aici: doar aplicația RAI Documente (profil `documents`) — UI + API Node + Postgres + uploads + Mistral OCR. Nu TMS full, nu OSRM, nu VROOM.

Cuprins

1. Arhitectură (cum stau piesele)
2. Înainte să începi (checklist)
3. Pregătire VPS (Ubuntu)
4. Postgres în Docker
5. Codul aplicației (clone branch prod)
6. Fișierul `.env` de producție
7. Primul build + migrare
8. systemd — API mereu pornit
9. Caddy — HTTPS + UI + proxy
10. Utilizator admin (prima oară)
11. Checklist go-live (înainte să dai linkul clientului)
12. Backup (pe VPS + snapshot Contabo)
13. Update la push (GitHub Actions)
14. Operațiuni uzuale & rollback
15. Probleme frecvente

1. Arhitectură



Scriptul din repo `npm run deploy:companion` face, în ordine: **backup** → **migrate** → **build companion** → **restart**. Nu inversa ordinea.

2. Înainte să începi

- ☐ DNS: `RAI-Transitix.filumforge.ro` → A → `185.137.122.45` (gata)
- ☐ Acces SSH root (sau user cu sudo) la VPS
- ☐ Repo GitHub: branch `TRANSITIX-RAI-PROD` există și e la commit-ul dorit
- ☐ Cheie **Mistral** Document AI (cont Mistral)
- ☐ Parolă tare pentru Postgres + `JWT_SECRET` (generezi pe VPS)
- ☐ Deploy key SSH (read) pe VPS pentru `git pull`, plus cheie în GitHub Secrets pentru Actions
- ☐ Snapshot Contabo gol / slot liber (plasă după setup)

> **Secrete:** nu pune niciodată `MISTRAL_API_KEY`, `JWT_SECRET` sau parola DB în Git. Doar pe VPS în `server/.env.companion` și în GitHub Environment secrets (pentru deploy SSH).

3. Pregătire VPS (Ubuntu 22.04 / 24.04)

Conectare (de pe PC):

```
ssh root@185.137.122.45
```

3.1 Update + pachete de bază

```
apt update && apt upgrade -y
apt install -y curl git ca-certificates gnupg ufw fail2ban \
  build-essential python3
```

3.2 Firewall

```
ufw default deny incoming
ufw default allow outgoing
ufw allow OpenSSH
ufw allow 80/tcp
ufw allow 443/tcp
ufw enable
ufw status
```

> **Nu deschide 3002 sau 5435 spre internet.** API și Postgres rămân pe localhost / Docker network.

3.3 Docker (doar pentru Postgres)

```
curl -fsSL https://get.docker.com | sh
systemctl enable --now docker
docker --version
docker compose version
```

3.4 Node.js 22 (LTS)

```
curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
apt install -y nodejs
node -v   # v22.x
npm -v
```

3.5 Caddy

```
apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' \
  | gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' \
  | tee /etc/apt/sources.list.d/caddy-stable.list
apt update && apt install -y caddy
caddy version
```

4. Postgres în Docker

Creează un director pentru datele companion (separat de home):

```
mkdir -p /opt/rai
cd /opt/rai
```

După ce ai clonat repo-ul (secțiunea 5), fișierul `docker-compose.companion.yml` din root pornește doar Postgres pe portul **5435**, baza `transitix_rai`.

Generează o parolă DB și pune-o într-un fișier env lângă compose (nu în Git):

```
openssl rand -base64 32
nano /opt/rai/TransitiX/.env.postgres
```

```
# /opt/rai/TransitiX/.env.postgres
POSTGRES_DB=transitix_rai
POSTGRES_USER=postgres
POSTGRES_PASSWORD=AICI_PAROLA_TARE_DIN_OPENSSL
```

Pornește baza (din root-ul repo-ului):

```
cd /opt/rai/TransitiX
docker compose -f docker-compose.companion.yml --env-file .env.postgres up -d
docker compose -f docker-compose.companion.yml ps
docker compose -f docker-compose.companion.yml exec -T db \
  pg_isready -U postgres -d transitix_rai
```

> Volumul Docker `transitix_rai_pg` păstrează datele la restart. Backup-ul aplicației (`pg_dump`) e separat — vezi §12.

5. Codul aplicației

5.1 Deploy key (read-only) pe VPS

```
ssh-keygen -t ed25519 -C "rai-prod-vps" -f /root/.ssh/rai_github -N ""
cat /root/.ssh/rai_github.pub
```

În GitHub → repo → **Settings** → **Deploy keys** → **Add** → lipește cheia publică (fără write).

```
cat > /root/.ssh/config <<'EOF'
Host github.com
  HostName github.com
  User git
  IdentityFile /root/.ssh/rai_github
  IdentitiesOnly yes
EOF
chmod 600 /root/.ssh/config
```

5.2 Clone branch de producție

```
cd /opt/rai
git clone git@github.com:Georgiana-Ghimes/TransitiX.git
cd TransitiX
git checkout TRANSITIX-RAI-PROD
git status
git log -1 --oneline
```

> **Branch-uri:** `MISTRAL-experimenting-rai-documents` = test pe PC; `TRANSITIX-RAI-PROD` = ce rulează aici. Pe VPS stai *doar* pe `TRANSITIX-RAI-PROD` .

5.3 Dependențe npm

```
cd /opt/rai/TransitiX
npm ci
npm ci --prefix server
```

6. Fișierul `.env` de producție

API-ul companion citește `server/.env`, apoi (dacă `COMPANION=1`) suprascrie cu `server/.env.companion`. Pe VPS folosim ambele: base minimal + companion complet.

```
openssl rand -hex 48   # → JWT_SECRET
```

```
nano /opt/rai/TransitiX/server/.env.companion
```

```
# server/.env.companion — PRODUCȚIE RAI (nu comita)
PORT=3002
DATABASE_URL=postgresql://postgres:AICI_PAROLA_TARE@127.0.0.1:5435/transitix_rai

JWT_SECRET=AICI_HEX_48_OCTETI
JWT_EXPIRES_IN=24h
JWT_REFRESH_EXPIRES_IN=7d

UPLOAD_DIR=uploads-companion
CLIENT_ORIGIN=https://rai-transitix.filumforge.ro
TRUST_PROXY=1

ALLOW_SIGNUP=false
GOOGLE_CLIENT_ID=

OCR_PROVIDER=mistral
MISTRAL_API_KEY=AICI_CHEIA_MISTRAL

PHOTON_URL=https://photon.komoot.io

NODE_ENV=production
```

Asigură-te că parola din `DATABASE_URL` = cea din `.env.postgres`.

Variabilă	De ce contează
`CLIENT_ORIGIN`	CORS + linkuri din email-uri. Exact domeniul HTTPS, fără slash final.
`TRUST_PROXY=1`	În spatele Caddy: IP real + HTTPS detectat corect. Fără proxy în față = lasă 0.
`ALLOW_SIGNUP=false`	Producție client: conturi doar prin invitație / manual admin.
`OCR_PROVIDER=mistral`	Fără cheie → OCR `none` și avizele nu se citesc.
`UPLOAD_DIR`	Folder relativ la `server/`. Intră în backup împreună cu DB.

```
chmod 600 /opt/rai/TransitiX/server/.env.companion
mkdir -p /opt/rai/TransitiX/server/uploads-companion
```

7. Primul build + migrare

```
cd /opt/rai/TransitiX
export COMPANION=1
export DATABASE_URL="$(grep ^DATABASE_URL= server/.env.companion | cut -d= -f2-)"

# migrare schemă pe transitix_rai
npm run db:migrate

# UI companion (profil documents) → dist/
npm run build:companion

ls -la dist/ | head
```

> Build-ul Vite cu `--mode companion` bake-uiește versiunea din `package.json → companion` (`__APP_VERSION__` / `__APP_BUILD__`).

8. systemd — API mereu pornit

```
nano /etc/systemd/system/rai-api.service
```

```
[Unit]
Description=RAI Transitix companion API
After=network.target docker.service
Requires=docker.service
Wants=docker.service

[Service]
Type=simple
User=root
WorkingDirectory=/opt/rai/TransitiX/server
Environment=COMPANION=1
Environment=NODE_ENV=production
# loadEnv citește .env apoi .env.companion când COMPANION=1
ExecStart=/usr/bin/node src/index.js
Restart=always
RestartSec=5
# nu rula migrate aici la fiecare crash-loop — migrate e în deploy

[Install]
WantedBy=multi-user.target
```

> **Notă:** `npm start` din `server/` rulează migrate la fiecare start. Pentru producție e mai sigur `node src/index.js` + migrate *doar* în `deploy-companion.js` (înainte de restart).

```
systemctl daemon-reload
systemctl enable --now rai-api
systemctl status rai-api --no-pager
journalctl -u rai-api -n 40 --no-pager
```

Verificare locală pe VPS:

```
curl -sS http://127.0.0.1:3002/api/health | head -c 400; echo
```

În răspuns, capacitatea OCR trebuie să arate `mistral` (nu `none` / `mistral-down`).

9. Caddy — HTTPS + UI + proxy

```
nano /etc/caddy/Caddyfile
```

```
rai-transitix.filumforge.ro {
        encode gzip

        # API + uploads → Node
        handle /api* {
                reverse_proxy 127.0.0.1:3002
        }
        handle /uploads* {
                reverse_proxy 127.0.0.1:3002
        }

        # UI static (build companion)
        handle {
                root * /opt/rai/TransitiX/dist
                try_files {path} /index.html
                file_server
        }
}
```

```
caddy validate --config /etc/caddy/Caddyfile
systemctl reload caddy
systemctl status caddy --no-pager
```

Test din exterior (de pe PC):

```
curl -I https://rai-transitix.filumforge.ro
curl -sS https://rai-transitix.filumforge.ro/api/health | head -c 400; echo
```

> Caddy obține singur certificatul Let's Encrypt dacă porturile 80/443 sunt deschise și DNS-ul pointează corect. Dacă TLS eșuează, verifică `journalctl -u caddy -n 50`.

10. Utilizator admin (prima oară)

Cu `ALLOW_SIGNUP=false`, creezi contul prin seed sau invitație.

Varianta A — seed (dacă scriptul din repo e potrivit pentru companion):

```
cd /opt/rai/TransitiX
COMPANION=1 npm run db:seed --prefix server
```

Notează email/parola afișate (sau din documentația seed). Schimbă parola imediat după login.

Varianta B — invitație după ce există deja un admin: din UI Setări / Users.

Autentifică-te pe `https://rai-transitix.filumforge.ro` — trebuie să vezi meniul companion (Avize, Rapoarte, Flotă, Ghid, Tichet suport), nu TMS full.

11. Checklist go-live

- ☐ `https://rai-transitix.filumforge.ro` încarcă login (padlock verde)
- ☐ `/api/health` OK; OCR = mistral
- ☐ Login admin; roluri corecte
- ☐ Upload un PDF test (1–2 pagini) → extracție → greutăți / TPO vizibile

- ☐ Tichet suport deschide formularul ClickUp
- ☐ Versiunea din UI = companion din `package.json` (build curent)
- ☐ `systemctl is-enabled rai-api caddy` → enabled
- ☐ Reboot de probă: `reboot` → după 2 min site-ul răspunde singur
- ☐ Backup de probă reușit (§12)
- ☐ Snapshot Contabo după go-live (§12.3)

12. Backup

12.1 Backup aplicație (DB + uploads) — pe VPS

Scriptul din repo: `npm run db:backup` → `scripts/backup-companion.js` (dump Postgres + arhivă uploads). Folosește `pg_dump` din container dacă lipsește pe host.

```
cd /opt/rai/TransitiX
COMPANION=1 DATABASE_URL="postgresql://postgres:…@127.0.0.1:5435/transitix_rai" \
  npm run db:backup -- --out /var/backups/rai --keep 14
```

Cron zilnic (ex. 02:15 Europe/Bucharest):

```
mkdir -p /var/backups/rai
crontab -e
```

```
15 2 * * * cd /opt/rai/TransitiX && COMPANION=1 /usr/bin/npm run db:backup -- --out /var/backups/rai --keep 14 >> /var/log/rai-backup.log 2>&1
```

Ce	Unde	Rol
pg_dump + uploads	`/var/backups/rai/`	Restaurare date / greșeală deploy
Backup la fiecare deploy	automat în `deploy-companion.js`	Înainte de migrate
Snapshot Contabo	panou Contabo	VPS / disc întreg

12.2 Restaurare rapidă (DB)

```
# oprește API ca să nu scrie în timpul restore
systemctl stop rai-api

# exemplu: dump SQL din backup
docker compose -f /opt/rai/TransitiX/docker-compose.companion.yml exec -T db \
  psql -U postgres -d transitix_rai < /var/backups/rai/….sql

# restoresază și folderul uploads din arhiva backup-ului
systemctl start rai-api
```

12.3 Snapshot Contabo

- După go-live reușit: **un snapshot** („rai-prod-golive”).
- Înainte de upgrade-uri majore de OS / Docker: încă unul.
- Nu înlocuiește backup-ul zilnic DB+uploads — e plasă pentru mașină moartă.

13. Update la push (GitHub Actions)

Flux: lucrezi pe `MISTRAL-…` → când e gata pentru client, **merge FF** în `TRANSITIX-RAI-PROD` (același hash pe ambele) → push → Actions deploy pe environment **RAI-PROD** / production.

13.1 Secrets pe environment (GitHub)

Settings → Environments → **RAI-PROD** (sau production) → Environment secrets:

Secret	Valoare
`VPS_HOST`	`185.137.122.45`
`VPS_USER`	`root` (sau userul tău)
`VPS_SSH_KEY`	cheia **privată** cu care Actions se conectează (ed25519)

Deployment branches: doar `TRANSITIX-RAI-PROD`.

13.2 Cheie SSH pentru Actions

Generează o pereche *separată* de deploy key-ul de pe VPS (sau folosește aceeași logică):

```
# pe PC (o dată)
ssh-keygen -t ed25519 -C "github-actions-rai-prod" -f rai-actions -N ""
# publică → /root/.ssh/authorized_keys pe VPS
# privată → secret VPS_SSH_KEY în environment
```

13.3 Fișier workflow (de adăugat în repo)

Cale: `.github/workflows/deploy-rai-prod.yml`

```
name: Deploy RAI PROD

on:
  push:
    branches: [TRANSITIX-RAI-PROD]
  workflow_dispatch:

concurrency:
  group: deploy-rai-prod
  cancel-in-progress: false

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: RAI-PROD
    steps:
      - name: Deploy over SSH
        uses: appleboy/ssh-action@v1.2.0
        with:
          host: ${{ secrets.VPS_HOST }}
          username: ${{ secrets.VPS_USER }}
          key: ${{ secrets.VPS_SSH_KEY }}
          script_stop: true
          script: |
            set -euo pipefail
            cd /opt/rai/TransitiX
            npm run deploy:companion:vps
            echo "deploy ok $(git rev-parse --short HEAD)"
```

> `deploy:companion:vps` = git → npm ci → backup → migrate → build → restart → health. Dacă un pas eșuează, Actions e roșu și vezi logul — clientul rămâne pe versiunea veche până repari.

13.4 Cum urci o versiune nouă (zi cu zi)

1. Pe PC: dezvoltare + test pe `MISTRAL-experimenting-rai-documents`.
2. Când e gata: checkout `TRANSITIX-RAI-PROD` → `git merge --ff-only MISTRAL-experimenting-rai-documents` → `git push` (același SHA pe ambele).
3. GitHub Actions → job Deploy → environment RAI-PROD → verde.
4. Verifică UI + health; build-ul companion din ecran.

14. Operațiuni uzuale & rollback

Status

```
systemctl status rai-api caddy --no-pager
docker compose -f /opt/rai/TransitiX/docker-compose.companion.yml ps
df -h
journalctl -u rai-api -n 80 --no-pager
```

Deploy manual (fără Actions)

```
cd /opt/rai/TransitiX
npm run deploy:companion:vps
```

git + npm ci + backup → migrate → build → restart + health.

Rollback cod (păstrează DB)

```
cd /opt/rai/TransitiX
git log --oneline -10
git reset --hard <commit-bun>
npm run deploy:companion:vps -- --skip-git --skip-backup
# pe GitHub: aliniază TRANSITIX-RAI-PROD la același SHA
```

Rollback date

Restore din `/var/backups/rai` (§12.2). Snapshot Contabo doar dacă mașina e compromisă.

15. Probleme frecvente

Simptom	Verifică
Site nu deschide / timeout	UFW 80/443; Caddy activ; DNS A corect; `curl -I` de pe VPS
502 Bad Gateway	`systemctl status rai-api`; API pe 3002; Caddy reverse_proxy
Login CORS / cookie	`CLIENT_ORIGIN=https://rai-transitix.filumforge.ro` exact
OCR nu citește	`MISTRAL_API_KEY`; health → mistral; loguri API
Migrate pe baza greșită	Mereu `--expect-db transitix_rai`; nu folosi DB-ul TMS full
UI vechi după deploy	`npm run build:companion` a rulat? hard refresh; Caddy root = `dist/`
Disk plin	`df -h`; prune uploads (`npm run uploads:prune`); retenție backup `--keep`

> **Capacitate VPS:** 4 vCPU / 8 GB / 100 GB e confortabil pentru companion + Postgres + OCR cloud. Nu instala OSRM pe această mașină.

Anexă — căi importante pe VPS

Cale	Conținut
`/opt/rai/TransitiX`	Repo (branch TRANSITIX-RAI-PROD)
`/opt/rai/TransitiX/dist`	UI build (servit de Caddy)
`/opt/rai/TransitiX/server/.env.companion`	Secrete producție
`/opt/rai/TransitiX/server/uploads-companion`	Fișiere client
`/var/backups/rai`	Backup-uri zilnice + la deploy
`/etc/caddy/Caddyfile`	HTTPS + proxy
`/etc/systemd/system/rai-api.service`	API persistent

Ghid deploy RAI Companion PROD · Contabo VPS 4 · rai-transitix.filumforge.ro · branch TRANSITIX-RAI-PROD · docs/VPS (local) · 12 octombrie 2026